```
# np.where(condition, True, False)
```

```
import numpy as np
marks = np.array([3,5,7,2,1])
```

```
marks
```

```
array([3, 5, 7, 2, 1])
```

```
np.where(marks > 5, "Yes", "No")
```

```
array(['No', 'No', 'Yes', 'No', 'No'], dtype='<U3')
```

```
Give students this towards the end.

employees = np.array([
    [101, 45000, 120000],
    [102, 60000, 180000],
    [103, 35000, 90000],
    [104, 75000, 250000],
    [105, 50000, 160000]
])

Columns:

Employee_ID | Salary | Sales
```

```
# Employees with sales >= ₹150,000 receive 5% commission otherwise 2% commision.

# 2. Sales > 200000, excellent, 150000 --> good, else improvement.
```

```
employees = np.array([
    [101, 45000, 120000],
    [102, 60000, 180000],
    [103, 35000, 90000],
    [104, 75000, 250000],
    [105, 50000, 160000]
])
```

```
sales = employees[:,2]
print(sales)
```

```
[120000 180000  90000 250000 160000]
```

```
category = np.where(sales > 200000, "Excellent", np.where(sales > 150000, "Good", "Needs Improvement"))
```

```
print(category)
```

```
['Needs Improvement' 'Good' 'Needs Improvement' 'Excellent' 'Good']
```

```
commision = np.where(sales >= 150000,sales*0.05,sales*0.02 )
```

```
print(commision)
```

```
[ 2400.  9000.  1800. 12500.  8000.]
```

```
import numpy as np

sales = np.array([
    120, 150, 180, 200,
    220, 250, 270, 300,
    320, 350, 380, 400
])

print(sales)
```

```
[120 150 180 200 220 250 270 300 320 350 380 400]
```

```
sales.ndim
```

```
1
```

```
sales.shape
```

```
(12,)
```

```
# 4 quarters --> 3 months each.
```

```
qtr_sales = sales.reshape(4,3)
qtr_sales
```

```
array([[120, 150, 180],
       [200, 220, 250],
       [270, 300, 320],
       [350, 380, 400]])
```

```
qtr_sales.shape
```

```
(4, 3)
```

```
sales
```

```
array([120, 150, 180, 200, 220, 250, 270, 300, 320, 350, 380, 400])
```

```
numbers = np.array([1, 2, 3, 4, 5, 6])
a = numbers.reshape(2,3)
print(a)
```

```
[[1 2 3]
 [4 5 6]]
```

```
numbers = np.array([1, 2, 3, 4, 5, 6])
a = numbers.reshape(3,3)
print(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_2561/343327727.py in <cell line: 0>()
      1 numbers = np.array([1, 2, 3, 4, 5, 6])
----> 2 a = numbers.reshape(3,3)
      3 print(a)

ValueError: cannot reshape array of size 6 into shape (3,3)
```

Next steps: Explain error

```
marks = np.array([
    72, 81, 65, 90, 77,
    88, 69, 91, 73, 84,
    60, 75, 82, 79, 95,
    68, 71, 86, 89, 93
])

# 4*5
# sum of each row
```

```
n_m = marks.reshape(4,5)
n_m
```

```
array([[72, 81, 65, 90, 77],
       [88, 69, 91, 73, 84],
       [60, 75, 82, 79, 95],
       [68, 71, 86, 89, 93]])
```

```
np.sum(n_m, axis = 1)
```

```
array([385, 405, 391, 407])
```

```
marks
```

```
array([72, 81, 65, 90, 77, 88, 69, 91, 73, 84, 60, 75, 82, 79, 95, 68, 71,
       86, 89, 93])
```

```
marks.shape
```

```
(20,)
```

```
n_m = marks.reshape(5,-1)
n_m
```

```
array([[72, 81, 65, 90],
       [77, 88, 69, 91],
       [73, 84, 60, 75],
       [82, 79, 95, 68],
       [71, 86, 89, 93]])
```

```
a = n_m.reshape(-1,2)
a
```

```
array([[72, 81],
       [65, 90],
       [77, 88],
       [69, 91],
       [73, 84],
       [60, 75],
       [82, 79],
       [95, 68],
       [71, 86],
       [89, 93]])
```

```
b = n_m.reshape(-1,3)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_2561/468334090.py in <cell line: 0>()
----> 1 b = n_m.reshape(-1,3)

ValueError: cannot reshape array of size 20 into shape (3)
```

Next steps: Explain error

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

```
matrix.shape
```

```
(3, 3)
```

```
a = matrix.flatten()
```

```
matrix.shape
```

```
(3, 3)
```

```
a.shape
```

```
(9,)
```

```
sales = np.array([
    [100, 120, 150],
    [90, 110, 130],
    [200, 220, 250]
])

# want the average sales for all 9 months
```

```
np.mean(sales)
```

```
np.float64(152.22222222222223)
```

```
np.mean(sales, axis = 1)
```

```
array([123.33333333, 110.        , 223.33333333])
```

```
np.mean(sales, axis = 0)
```

```
array([130.        , 150.        , 176.66666667])
```

```
a = sales.flatten()
```

```
np.mean(a)
```

```
np.float64(152.22222222222223)
```

```
a = np.arange(30)
print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29]
```

```
b = a.reshape(2,5,3)
print(b)
```

```
[[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]
  [ 9 10 11]
  [12 13 14]]

 [[15 16 17]
  [18 19 20]
  [21 22 23]
  [24 25 26]
  [27 28 29]]]
```

```
b[0]
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14]])
```

```
b[0,1]
```

```
array([3, 4, 5])
```

DPP

```
# An e-commerce company records the quantity sold and selling price of 12 products.

quantity = np.array([2, 5, 3, 8, 4, 6, 10, 1, 7, 5, 9, 3])

price = np.array([
    1200, 800, 1500, 450, 2200, 900,
    700, 1800, 650, 1300, 500, 2500
])
```

```
Calculate revenue for every product.
Calculate the total revenue.
Find the product index generating the highest revenue.
Extract the revenue of the last five products.
```

```
rev = quantity*price
(rev)
```

```
array([2400, 4000, 4500, 3600, 8800, 5400, 7000, 1800, 4550, 6500, 4500,
       7500])
```

```
# Calculate the total revenue.
np.sum(rev)
```

```
np.int64(60550)
```

```
# Find the product index generating the highest revenue.
np.argmax(rev)
```

```
np.int64(4)
```

```
# Extract the revenue of the last five products.

rev[-5:]
```

```
array([1800, 4550, 6500, 4500, 7500])
```

```
Find products whose revenue is above the average revenue.
Apply a 10% discount only to products whose price is greater than ₹1,000.
Round the discounted prices to two decimal places.
Calculate the total quantity sold.
Find the median product price.
Identify products whose price is above the median and whose quantity is at least 5.
```

```
avg_rev = np.mean(rev)
avg_rev
```

```
np.float64(5045.833333333333)
```

```
rev[rev > avg_rev]
```

```
array([8800, 5400, 7000, 6500, 7500])
```

```
np.where(rev > avg_rev, "Yes", "No")
```

```
array(['No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'No', 'Yes',
       'No', 'Yes'], dtype='<U3')
```

```
# Apply a 10% discount only to products whose price is greater than ₹1,000.


price
```

```
array([1200,  800, 1500,  450, 2200,  900,  700, 1800,  650, 1300,  500,
       2500])
```

```
discounted_p = np.where(price > 1000, 0.9*price, price)
print(discounted_p)
```

```
[1080.  800. 1350.  450. 1980.  900.  700. 1620.  650. 1170.  500. 2250.]
```

```
# Round the discounted prices to two decimal places.
discounted_p = np.round(discounted_p,2)
discounted_p
```

```
array([1080.,  800., 1350.,  450., 1980.,  900.,  700., 1620.,  650.,
       1170.,  500., 2250.])
```

```
# Calculate the total quantity sold.
np.sum(quantity)
```

```
np.int64(63)
```

```
# Find the median product price.
med_p = np.median(price)
```

```
# Identify products whose price is above the median and whose quantity is at least 5.

criteria = (price > med_p) & (quantity >= 5)
price[criteria]
```

```
array([1300])
```

A college has performance scores for 24 assessments:

```
scores = np.array([
    72, 81, 65, 90,
    77, 88, 69, 91,
    73, 84, 60, 75,
    82, 79, 95, 68,
    71, 86, 89, 93,
    66, 74, 80, 87
])
```

Assume there are 6 students and 4 assessments per student.

Verify the number of elements.
Reshape the array into a 6 × 4 matrix.
Calculate the average score for each student.
Find the highest score achieved by each student.

```
scores = np.array([
    72, 81, 65, 90,
    77, 88, 69, 91,
    73, 84, 60, 75,
    82, 79, 95, 68,
    71, 86, 89, 93,
    66, 74, 80, 87
])
```

```
np.size(scores)
```

```
24
```

```
# Reshape the array into a 6 × 4 matrix.
new_s = scores.reshape(6,4)
```

```
new_s
```

```
array([[72, 81, 65, 90],
       [77, 88, 69, 91],
       [73, 84, 60, 75],
       [82, 79, 95, 68],
       [71, 86, 89, 93],
       [66, 74, 80, 87]])
```

```
# Calculate the average score for each student.
new_s
```

```
array([[72, 81, 65, 90],
       [77, 88, 69, 91],
       [73, 84, 60, 75],
       [82, 79, 95, 68],
       [71, 86, 89, 93],
       [66, 74, 80, 87]])
```

```
np.mean(new_s, axis = 1)
```

```
array([77.  , 81.25, 73.  , 81.  , 84.75, 76.75])
```

```
# Find the highest score achieved by each student.

np.max(new_s, axis = 1)
```

```
array([90, 91, 84, 95, 93, 87])
```

Find the student index having the highest overall average.

Create performance labels:
average >= 85 → "Excellent"
average >= 70 → "Good"
average >= 50 → "Pass"
otherwise → "Fail"

```
Extract students whose average is above the overall average.
```

```
# Find the student index having the highest overall average.
avg_st = np.mean(new_s, axis = 1)
```

```
a = np.argmax(avg_st)
print(a)
```

```
4
```

```
np.where(avg_st >= 85, "Excellent", np.where(avg_st >= 70, "Good", np.where(avg_st >= 50, "Pass", "Fail")))
```

```
array(['Good', 'Good', 'Good', 'Good', 'Good', 'Good'], dtype='<U9')
```

```
# Extract students whose average is above the overall average.

scores
```

```
array([72, 81, 65, 90, 77, 88, 69, 91, 73, 84, 60, 75, 82, 79, 95, 68, 71,
       86, 89, 93, 66, 74, 80, 87])
```

```
avg_sc = np.mean(scores)
```

```
ans = np.where(avg_st > avg_sc, "Yes", "No")
```

```
ans
```

```
array(['No', 'Yes', 'No', 'Yes', 'Yes', 'No'], dtype='<U3')
```

```
# Extract the last two assessments of every student.
# Flatten the complete matrix.
# Calculate the percentage of all assessment scores that are above 80.
```

```
new_s
```

```
array([[72, 81, 65, 90],
       [77, 88, 69, 91],
       [73, 84, 60, 75],
       [82, 79, 95, 68],
       [71, 86, 89, 93],
       [66, 74, 80, 87]])
```

```
new_s[:, -2:]
```

```
array([[65, 90],
       [69, 91],
       [60, 75],
       [95, 68],
       [89, 93],
       [80, 87]])
```

```
# Flatten the complete matrix.
scores.flatten()
```

```
array([72, 81, 65, 90, 77, 88, 69, 91, 73, 84, 60, 75, 82, 79, 95, 68, 71,
       86, 89, 93, 66, 74, 80, 87])
```

```
# Calculate the percentage of all assessment scores that are above 80.

grt_80 = scores[scores > 80]
grt_80
```

```
array([81, 90, 88, 91, 84, 82, 95, 86, 89, 93, 87])
```

```
a = len(grt_80)
n = len(scores)
perc = a*100/n
print(perc)
```

```
45.833333333333336
```

```
A company's monthly sales for one year are:

sales = np.array([
    120000, 135000, 128000, 150000,
    160000, 172000, 165000, 180000,
    195000, 210000, 205000, 225000
])
```

```
Calculate the average monthly sales.
Find the median sales.
Find the month with the highest sales.
Calculate the standard deviation.
```

```
sales = np.array([
    120000, 135000, 128000, 150000,
    160000, 172000, 165000, 180000,
    195000, 210000, 205000, 225000
])
```

```
np.mean(sales)
```

```
np.float64(170416.66666666666)
```

```
np.median(sales)
```

```
np.float64(168500.0)
```

```
# Find the month with the highest sales.
np.argmax(sales)
```

```
np.int64(11)
```

```
# Calculate the standard deviation.

np.std(sales)
```

```
np.float64(32392.021479919335)
```

```
Create a forecast for the next 6 months using np.linspace() between:
the minimum observed sales
and 20% above the maximum observed sales.

Round the forecast values to the nearest rupee.

Combine the original sales and forecast into one 18-element array.

Reshape the combined array into 3 × 6.

Calculate the average sales for each row.

Identify values that are greater than the overall median.
```

```
mn_s = np.min(sales)
mx_s = np.max(sales)
```

```
mx_s20 = mx_s*1.2
```

```
forecast = np.linspace(mn_s, mx_s20,6)
```

```
forecast
```

```
array([120000., 150000., 180000., 210000., 240000., 270000.])
```

```
# Round the forecast values to the nearest rupee.
r_s = np.round(forecast,2)
```

```
r_s
```

```
array([120000., 150000., 180000., 210000., 240000., 270000.])
```

```
sales
```

```
array([120000, 135000, 128000, 150000, 160000, 172000, 165000, 180000,
       195000, 210000, 205000, 225000])
```

```
n_s = np.append(sales, r_s)
n_s
```

```
array([120000., 135000., 128000., 150000., 160000., 172000., 165000.,
       180000., 195000., 210000., 205000., 225000., 120000., 150000.,
       180000., 210000., 240000., 270000.])
```

```
# Reshape the combined array into 3 × 6.
ans = n_s.reshape(3,6)
ans
```

```
array([[120000., 135000., 128000., 150000., 160000., 172000.],
       [165000., 180000., 195000., 210000., 205000., 225000.],
       [120000., 150000., 180000., 210000., 240000., 270000.]])
```

```
np.mean(ans, axis = 1)
```

```
array([144166.66666667, 196666.66666667, 195000.        ])
```

```
np.where(n_s > np.median(n_s), "Yes", "No")
```

```
array(['No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes',
       'Yes', 'Yes', 'No', 'No', 'Yes', 'Yes', 'Yes', 'Yes'], dtype='<U3')
```

HW

```
A factory collects temperatures from 20 sensor readings:

temperature = np.array([
    28.5, 31.2, 29.8, 35.6, 38.1,
    40.2, 27.9, 30.5, 42.3, 39.8,
    36.4, 33.2, 29.1, 41.7, 44.2,
    31.8, 34.5, 37.6, 26.9, 43.1
])
```

```
Calculate mean, median, minimum and maximum.
Calculate variance and standard deviation.
Find all temperatures more than one standard deviation above the mean.
Calculate the absolute difference between every temperature and the mean.
Find the five readings having the largest deviation from the mean.
Round all temperatures to the nearest whole number.
```